S08 Save and Load

CRE132 worksheet

What you'll make

Things that are still there next time the game runs. The volume sliders remember where you left them, a **Best** score keeps the highest score ever, and **Save**, **Load** and **Delete Save** buttons store your progress (position, health, score and collected coins) in a file.

By the end you'll know two ways to save: **PlayerPrefs** for settings and a few numbers, and a **JSON file** for game progress. You'll also know why a save file holds only data, not GameObjects.

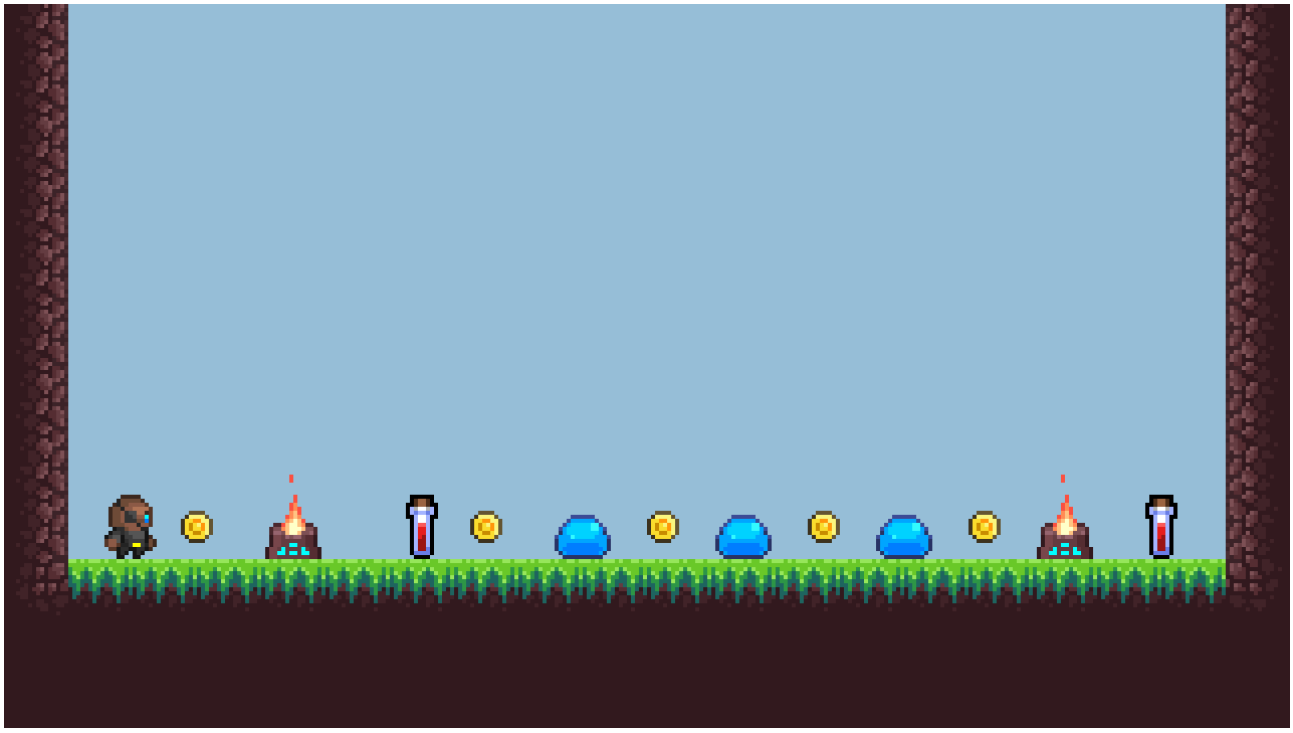


Figure 1: The finished scene: the player, coins, two fires, two potions and three training dummies.

Before you start

1. Unzip `CRE132_S08_SaveLoad_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S08_SaveLoad_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S08_SaveLoad`.
3. Press **Play**. It's S05's room: music, coins, fire, potions, health bar, score and volume sliders all work. Move a slider, stop and press Play again: it's back at full. **Best: 0** never changes, and **Save**, **Load** and **Delete Save** do nothing. Press Play again to stop.

The coins are called **Coin 1** to **Coin 5** (inside **Coins**). There's an empty **Save Game** object, and a **High Score Panel** under the score. The buttons' **On Click ()** lists are empty: you connect them to the save script.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Saving settings: PlayerPrefs in the AudioManager

PlayerPrefs is a small store of named values: `PlayerPrefs.SetFloat("MusicVolume", value)` saves a number under a name (a **key**), and `PlayerPrefs.GetFloat("MusicVolume", 1f)` reads it back. The second number is used when nothing has been saved yet. Unity writes PlayerPrefs to disk when the game quits.

Open `AudioManager` (`Scripts/Managers`). Add these lines at the top of `Start`, before `if (startingMusic != null)`:

```
        // Saved volumes from last time. The mixer ignores SetFloat in Awake, so this
        // has to be in Start.
        ApplyVolume("MusicVolume", GetVolume("MusicVolume"));
        ApplyVolume("SFXVolume", GetVolume("SFXVolume"));
```

Then replace everything from the `// Volume from a slider` comment above `SetVolume` to the end of the class (including the `GetVolume` you added in S06) with these three methods. Keep the class's last `}`.

```
    // Volume from a slider (0 to 1). It is saved in PlayerPrefs, so it's the same
    // next time the game runs.
    public void SetVolume(string exposedParameter, float sliderValue)
    {
        PlayerPrefs.SetFloat(exposedParameter, sliderValue);
        ApplyVolume(exposedParameter, sliderValue);
    }

    // The saved slider value, or 1 (full volume) if nothing has been saved yet.
    public float GetVolume(string exposedParameter)
    {
        return PlayerPrefs.GetFloat(exposedParameter, 1f);
    }

    // The mixer works in decibels (-80 to 0), so convert the 0 to 1 slider value.
    private void ApplyVolume(string exposedParameter, float sliderValue)
    {
        // Log10(0) can't be worked out, so keep the slider value just above zero.
        if (sliderValue < 0.0001f)
        {
            sliderValue = 0.0001f;
        }
        float decibels = Mathf.Log10(sliderValue) * 20f;
        mixer.SetFloat(exposedParameter, decibels);
    }
```

- **`SetVolume`** now saves the slider value under the exposed parameter's name (`MusicVolume` or `SFXVolume`), then hands it to `ApplyVolume`, which does the decibel maths from S05.
- **`GetVolume`** reads the saved value instead of asking the mixer, so it works even before the mixer has been set. VolumeSlider already calls it for its starting position, so it needs no changes.
- **`ApplyVolume` is private:** only the AudioManager needs it.
- **Why `Start`?** The mixer ignores `SetFloat` in `Awake`, a known Unity quirk.

Test it: press Play, move both sliders, stop, and press Play again. They're where you left them, and so is the volume.

2. A high score in the GameManager

Open `GameManager`. Add a field below `score`:

```
    // The best score ever, kept in PlayerPrefs so it survives quitting the game.
    [SerializeField] private int highScore;
```

and a property below the `Score` property:

```
    public int HighScore
    {
        get { return highScore; }
    }
```

At the end of `Awake`, below `DontDestroyOnLoad(gameObject);`, load the saved value:

```
        // 0 if nothing has been saved yet.
        highScore = PlayerPrefs.GetInt("HighScore", 0);
```

Then replace `AddScore` with this version, plus a new `SetScore` below it for loading a save:

```
    public void AddScore(int amount)
    {
        score = score + amount;
        Debug.Log("Score: " + score);

        if (score > highScore)
        {
            highScore = score;
            PlayerPrefs.SetInt("HighScore", highScore);
        }
    }

    // For loading a saved game.
    public void SetScore(int newScore)
    {
        score = newScore;
    }
```

- **The high score is loaded once**, in `Awake`, and saved whenever the score beats it.
- **PlayerPrefs is for settings and a few numbers.** A whole game's progress, with lists and positions, belongs in a save file (part 4).

Now show it. Create `HighScoreDisplay` in `Scripts/UI`:

```
// Shows the GameManager's high score ("Best: 12"), updating the text only when it
// changes.
using TMPro;
using UnityEngine;

public class HighScoreDisplay : MonoBehaviour
{
    [SerializeField] private TMP_Text highScoreText;

    private int shownHighScore = -1;

    private void Update()
    {
        if (GameManager.Instance == null)
        {
            return;
```

```
        }

        int highScore = GameManager.Instance.HighScore;
        if (highScore != shownHighScore)
        {
            shownHighScore = highScore;
            highScoreText.text = "Best: " + highScore;
        }
    }
}
```

It's S03's ScoreDisplay again, reading `HighScore` instead of `Score`.

Add it to **High Score Panel** and drag its child **Text** into **High Score Text**.

Test it: collect three coins, stop, and press Play: **Best: 3** is still there, even though the score is back to 0.

3. Setting health: a small change to Health

Loading a save needs to put the player's health back. Open `Health` (`Scripts/Health`) and add this method below `Heal`:

```
    // Sets health directly, for loading a saved game. Kept between 1 and the maximum.
    public void SetHealth(int amount)
    {
        currentHealth = Mathf.Clamp(amount, 1, maxHealth);
    }
```

`Mathf.Clamp` keeps a number between a minimum and a maximum: here at least 1 (loading shouldn't kill you) and at most `maxHealth`.

4. What goes in the file: SaveData

Make a folder `Assets/_game/Scripts/S08_SaveLoad` and create a script `SaveData` in it. This one isn't a MonoBehaviour, so replace everything Unity wrote:

```
// Everything that goes in the save file. JsonUtility turns the public fields into
// JSON text and back.
// [System.Serializable] is what lets JsonUtility (and the Inspector) read the fields.
using System.Collections.Generic;

[System.Serializable]
public class SaveData
{
    public float playerX;
    public float playerY;
    public int health;
    public int score;

    // The names of the coins already picked up, so they stay gone after loading.
    public List<string> collectedCoins = new List<string>();
}
```

- **A plain class:** no `: MonoBehaviour`, so it can't go on an object. It's just a bundle of data.
- **Only data goes in a save file:** numbers, text and lists, not GameObjects or components. You save what you need to rebuild the game, such as the player's x and y.
- **`[System.Serializable]`** lets `JsonUtility` read and write the public fields.

5. Saving and loading: SaveGame

Create `SaveGame` in the `S08_SaveLoad` folder:

```
// Saves progress to a JSON file and loads it back. Buttons call Save, Load and
// DeleteSave.
// Loading reloads the scene, and Start puts the saved progress back into the fresh
// scene.
using System.Collections.Generic;
using System.IO;
using UnityEngine;
using UnityEngine.SceneManagement;

public class SaveGame : MonoBehaviour
{
    [SerializeField] private Transform player;
    [SerializeField] private Health playerHealth;

    // The parent of every coin in the level. Each coin needs a different name.
    [SerializeField] private Transform coinsParent;

    private string savePath;
    private List<string> allCoinNames = new List<string>();

    private void Awake()
    {
        // persistentDataPath is a folder Unity can always write to, on any platform.
        savePath = Path.Combine(Application.persistentDataPath, "save.json");
    }

    private void Start()
    {
        // Remember every coin the level starts with, so Save can work out which ones
        // are gone.
        foreach (Transform coin in coinsParent)
        {
            allCoinNames.Add(coin.name);
        }

        if (File.Exists(savePath))
        {
            ApplySave();
        }
    }

    public void Save()
    {
        SaveData data = new SaveData();
        data.playerX = player.position.x;
        data.playerY = player.position.y;
        data.health = playerHealth.CurrentHealth;
        data.score = GameManager.Instance.Score;

        foreach (string coinName in allCoinNames)
        {
            if (coinsParent.Find(coinName) == null)
            {
                data.collectedCoins.Add(coinName);
            }
        }
```

```
        // true = "pretty print": spread over several lines, so the file is easy to
        // read.
        string json = JsonUtility.ToJson(data, true);
        File.WriteAllText(savePath, json);
        Debug.Log("Saved to " + savePath);
    }

    public void Load()
    {
        // A clean copy of the level; Start then applies the save file.
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }

    public void DeleteSave()
    {
        if (File.Exists(savePath))
        {
            File.Delete(savePath);
            Debug.Log("Save deleted");
        }
        GameManager.Instance.ResetScore();
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }

    private void ApplySave()
    {
        string json = File.ReadAllText(savePath);
        SaveData data = JsonUtility.FromJson<SaveData>(json);

        player.position = new Vector3(data.playerX, data.playerY, 0f);
        playerHealth.SetHealth(data.health);
        GameManager.Instance.SetScore(data.score);

        foreach (string coinName in data.collectedCoins)
        {
            Transform coin = coinsParent.Find(coinName);
            if (coin != null)
            {
                Destroy(coin.gameObject);
            }
        }
        Debug.Log("Loaded " + savePath);
    }
}
```

What's going on:

- **`Application.persistentDataPath`** is a folder Unity can always write to, on any platform. `Path.Combine` adds the file name with the right slash for the system.
- **Saving:** fill a `SaveData`, turn it into text with `JsonUtility.ToJson`, and write it with `File.WriteAllText`.
- **Coins are saved by name.** `Start` remembers every coin's name; `Save` uses `coinsParent.Find` to see which ones are missing (collected). Two coins with the same name would get mixed up, which is why they're numbered.
- **Loading reloads the scene** for a clean level, then `Start` reads the file with `File.ReadAllText` and `JsonUtility.FromJson<SaveData>`, and puts everything back: position, health, score, and destroys the coins already collected.
- **`File.Exists`** comes first: the first time the game runs, there's no save yet.

Add it to **Save Game**. Drag the **Player** into both **Player** and **Player Health**, and **Coins** into **Coins Parent**. Then wire the three buttons: click **+** in each one's On Click () list, drag **Save Game** into the new entry's object slot and pick **SaveGame > Save**, **SaveGame > Load** and **SaveGame > DeleteSave**.

Test it: collect a coin, lose some health in the fire, walk somewhere and press **Save**. Wander off and press **Load**: you're back where you saved, with the same health, score and coins. Stop and press Play: it loads the save again. **Delete Save** starts afresh.

The Console shows where the file is. On Linux that's `~/.config/unity3d/DefaultCompany/UnityTuts/save.json`. Open it in a text editor, change the score to 99, save the file and press **Load**. It's just text, which is also why players can cheat with it.

Check your work

- ☐ The volume sliders are where you left them after stopping and pressing Play.
- ☐ **Best** keeps the highest score, even after restarting Unity.
- ☐ Save, then Load, puts back your position, health, score and coins.
- ☐ Delete Save starts afresh with a score of 0.
- ☐ Compare with `CRE132_S08_SaveLoad_Finished` if something behaves differently.

Key points

- **PlayerPrefs for settings and small values; a JSON file for game progress.**
- **Only data goes in a save file:** numbers, text and lists, not GameObjects or components. Save what you need to rebuild the state.
- **Check `File.Exists` before reading.** The first time the game runs, there's no save yet.

If something goes wrong

- **"The name 'ApplyVolume' does not exist":** you added the lines to `Start` but not the three new methods.
- **"Type 'AudioManager' already defines a member called 'GetVolume'":** the old S06 `GetVolume` is still at the bottom of the AudioManager. Delete it.
- **NullReferenceException in SaveGame:** the Player, Player Health or Coins Parent slot is empty.
- **Collected coins come back after loading: Coins Parent** must be **Coins**, the parent of the coins, not one coin.
- **A button does nothing:** its On Click () slot still says **None**, or the dropdown says **No Function**.
- **The game starts somewhere odd every time:** an old save is loading. Press **Delete Save**.

Try it

1. Save automatically when the player touches a checkpoint: a trigger that calls `Save()`.
2. Add the dummies' health to the save file.
3. Add a **Reset Settings** button that calls `PlayerPrefs.DeleteAll()`.